«Свет по видео» — описание проекта

Версия: 0.10.0 · **Дата:** 03.11.2026 · **Автор:** Bezuglyj E.N. (techlan.su)
Проект: `projects/light-by-video/` · **Память:** `memory/project-light-by-video.md`

---

1. Что это и зачем

Интеграция Home Assistant **«Свет по видео»** (`light_by_video`) включает выбранное устройство — свет, реле,
прожектор — **на заданное время**, когда в **нарисованной на кадре камеры области** появляется движение.
По истечении времени тишины устройство выключается; движение продлевает удержание; есть жёсткий лимит,
пауза между включениями, окно времени/дни и условия «только после заката».

Задача появилась так (слова владельца): «на камере нужно выбрать определённую область и при нахождении в ней
движения включать на время устройство, например свет». Пилот — гостевой дом площадки «Грозовая»:
камера **«Гостевой 13»** → выключатель **`switch.svv_grozovaia_grozovaia_gostevoi_dom_switch`**.

Отличие от «просто автоматизации»: область задаётся мышью на живом кадре, есть журнал решений и причины пропусков,
защита от ложных срабатываний (смена освещения, животные, припаркованные машины), устойчивость к перезапуску HA.

---

2. Состав проекта

Часть	Где	Что делает
Интеграция HA `light_by_video`	`/config/custom_components/light_by_video/` (канон — `custom_components/` проекта)	опрос камеры, детекция, действия, панель, сущности, HTTP API
Детектор объектов (опция)	ARM `192.168.103.202:8099`, `/opt/light-by-video-detector`, systemd `lbv-detector`	YOLOv5n ONNX (CPU): JPEG → bbox человека/машины/…
Документы	`docs/`	это описание, FAQ (PDF/XLSX), план
Тесты	`tools/test_motion.py`	офлайн-проверка детектора на

Часть	Где	Что делает
		синтетических кадрах (12 проверок)

Почему детектор объектов — отдельный сервис: контейнер Home Assistant построен на Alpine (musl) с Python 3.14,
а у `onnxruntime` и `opencv` **нет musllinux-колёс** — внутрь HA их не установить без сборки из исходников.
Поэтому CPU-инференс вынесен на ARM (Debian, glibc), HA обращается к нему по HTTP.

---

3. Архитектура и поток данных

```
камера (RTSP Line-сервера 192.168.1.250, канал 12)
        │  субпоток HEVC 640×360, 8 fps
        ▼
┌──────────────────────── Home Assistant «База» ────────────────────────┐
│ light_by_video                                                        │
│  источник:  RTSP (PyAV) │ HTTP-снапшот │ camera.*                     │
│      │                                                                │
│      ▼                                                                │
│  кадр → серый 480 px → сглаживание → разность с фоном (EMA)           │
│      │                                    │                           │
│      │ (режим «объекты»)                  │ метрики зон: доля изменённых │
│      ▼                                    │ пикселей, px, bbox        │
│  POST /detect → YOLOv5n (ARM:8099)        │                           │
│      │ bbox person/car/…                  │                           │
│      └──────────────┬─────────────────────┘                           │
│                     ▼                                                 │
│     подтверждение 2 из 3 кадров → «движение в зоне»                   │
│                     ▼                                                 │
│     действия: turn_on → таймер hold → продление → turn_off (лимит)    │
│                     ▼                                                 │
│     сущности (binary_sensor/sensor/switch), события шины, журнал      │
└───────────────────────────────────────────────────────────────────────┘
```

Составляющие модуля интеграции:

Файл	Роль
`const.py`	домен, ключи настроек, значения по умолчанию, события, URL API
`config_flow.py`	мастер добавления: камера HA / RTSP / URL / поток go2rtc, интервал, проверка кадра
`settings.py`	нормализация и хранение настроек в `config_entry.options`, кэш в

Файл	Роль
	памяти
`sources.py`	источники кадра: `HttpSource`, `CameraSource`, `RtspSource` (PyAV + сторож переподключения)
`motion.py`	детектор «разность кадров»: фон EMA, маска, чистка, метрики по зонам, PNG-маска для отладки
`coordinator.py`	цикл опроса, временная логика зон, вызов детектора объектов, события, журнал
`actions.py`	включение/удержание/выключение, продление, лимиты, пауза, восстановление таймеров после рестарта
`binary_sensor.py` / `sensor.py` / `switch.py`	сущности HA
`views.py`	HTTP API для панели и внешних инструментов
`panel.py` + `frontend/panel.js`	панель «Свет по видео» в боковом меню

---

4. Пилот (фактическая конфигурация)

Параметр	Значение
HA	«База», Core 2026.10.4, API `https://192.168.103.200:38123` (TLS)
Запись интеграции	`01M6FWPTKCG2XG7V5M70FVTT8Z`, заголовок «Гостевой 13»
Камера	«Гостевой 13» = **`/cameras/12`** на Line-сервере `192.168.1.250` (логин `admin`)
Поток	`rtsp://admin:***@192.168.1.250:9784/cameras/12/streaming/sub` (HEVC 640×360, 8 fps)
Устройство	`switch.svv_grozovaia_grozovaia_gostevoi_dom_switch`
Зона	«Двор — вход в гостевой дом», x/y/w/h в долях кадра
Действие	держать 300 с, продление, максимум 1800 с, пауза 300 с
Условие	**только после заката** (`sun.sun` = below_horizon)

Параметр	Значение
Детектор	режим «объекты», порог 0.3, классы person/bicycle/car/motorcycle/bus/truck, «только вместе с движением»

Проверено: днём действие корректно пропускается («до заката (светло)»), свет остаётся выключен; детектор отвечает
244–357 мс на кадр; припаркованная машина не даёт срабатывания (объект есть, движения нет).

---

5. Установка

5.1. Интеграция в Home Assistant

1. Скопировать каталог `custom_components/light_by_video` в `/config/custom_components/`.
2. Перезапустить ядро HA (Настройки → Система → Перезапустить; либо `ha core restart`).
3. **Настройки → Устройства и службы → Добавить интеграцию → «Свет по видео»**:
   - камера Home Assistant (если заведена) — приоритетный источник; **или**
   - поток: имя go2rtc, `rtsp://…`, `http://…/image`;
   - интервал опроса (1–30 с).
   Мастер проверяет кадр и не даст сохранить нерабочий источник.
4. Открыть панель **«Свет по видео»**, нарисовать область, выбрать устройство.

5.2. Детектор объектов (если нужен режим «объекты»)

```
# на ARM
mkdir -p /opt/light-by-video-detector/models && cd /opt/light-by-video-detector
python3 -m venv venv
./venv/bin/pip install onnxruntime numpy pillow aiohttp onnx
# модель yolov5n.onnx (FP32) → models/yolov5n_fp32.onnx
cp service.py . && cp detector/lbv-detector.service /etc/systemd/system/
systemctl daemon-reload && systemctl enable --now lbv-detector
curl -s http://127.0.0.1:8099/health
```

В интеграции: вкладка «Детектор» → режим **«объекты»**, адрес
`http://192.168.103.202:8099/detect`.

---

6. Настройка в панели

Вкладка	Что настраивается
Зоны	рисование мышью (новая зона — протяжка по кадру), 4 угла для прямоугольника, **произвольный многоугольник**: кнопка «⬡ В многоугольник», перетаскивание вершин, добавление вершины по пунктиру на ребре, удаление правым кликом; точные X/Y/Ш/В, роль (`движение` / `исключить`), чувствительность, сущность зоны
Действия	**«Не беспокоить» 1/4/8 ч**, блок **«Состояние действия»** (включено/ожидание/пауза, осталось держать, до лимита, состояние устройства, время последней команды), блок **«Расписание»** (сейчас ночь/день, закат/восход, окно, дни, поправка), устройство, «держать»/«максимум»/пауза в минутах, продление, «не гасить, если горело», время суток, окно времени и дни, кнопки «Включить на 10 с», «Выключить сейчас», «Сбросить паузу»
Детектор	режим (разность кадров / объекты), **пресеты «Улица / Двор / Помещение»**, адрес сервиса, порог, классы COCO, «только вместе с движением», **дробление кадра для дальних объектов**, ширина кадра детектора, фильтр мелких объектов, пороги разности, «Показать маску»
Запреты	правила «сущность + условие → запрещать / предупреждать»: ворота открыты, свет уже горит, на улице светло и т.п. Видно, какое правило срабатывает сейчас
Источник	камера HA из списка либо RTSP/URL/go2rtc, интервал проверки, «Применить источник»
Журнал	события и причины пропусков, локальное время
Справка	полное описание функций в самой панели: зоны, действия, детектор, запреты, расписания, источник, формулы калибровки, сущности и сервисы

---

7. Детекция: как работает и как подбирать

7.1. Режим «разность кадров» (по умолчанию, без зависимостей)

1. Кадр уменьшается до 480 px по ширине и переводится в серый; сглаживание (`blur`) давит шум ИК/JPEG.
2. Держится фон: `bg = bg·(1−α) + кадр·α` (`bg_alpha` ≈ 0.03). Сравнение: `|кадр − bg| > diff_t` (`diff_t_ir` для ч/б).
3. Маска чистится морфологией (Min/Max 3×3); для каждой зоны считается доля изменённых пикселей и их количество.
   Зона может быть прямоугольником или **многоугольником** — тогда маска зоны строится заливкой полигона (PIL),
   а попадание объекта проверяется «точка в полигоне» + перекрытие площади по

сетке 14×14.

4. Срабатывание: `доля ≥ min_ratio / чувствительность` **и** `пикселей ≥ min_px` в `confirm_frames` из `confirm_window`.
   Окончание: `quiet_frames` подряд ниже порога.
5. Защита от смены освещения: если изменилось больше `global_change_ratio` (55 %) кадра — кадр помечается
   «смена света» (фары, рассвет, автоэкспозиция), состояние зон не меняется, фон подтягивается быстрее.

Ориентиры (реальный поток «Гостевого 13»): пустая сцена — 0 %, проход человека/машины — 8–10 %;
порог по умолчанию 1.5 %, `min_px` 150, `diff_t` 26, `blur` 1.

7.2. Режим «объекты» (ONNX, CPU)

Кадр отправляется в сервис `/detect`; возвращаются bbox разрешённых классов. Зона считается «занятой», если
центр объекта внутри зоны или объект перекрывает её более чем на 30 %. По умолчанию включено
«только вместе с движением» — объект подтверждается лишь при ненулевом изменении кадра в зоне.
Это отсекает главный ложный сценарий: припаркованная машина в зоне (объект есть постоянно, движения нет).

Инференс выполняется **только когда в кадре есть движение** (`objects_only_on_motion`): без движения результат
берётся из кэша (по умолчанию 3 с), поэтому в статичной сцене CPU почти не тратится.
Для **дальних объектов** кадр дробится: `objects_tiling=2` — сначала весь кадр, если ничего не нашли — 2×2 фрагмента
с перекрытием 12 % (каждый фрагмент подаётся в модель как отдельный кадр 640×640); `3` — дробить всегда (медленно).
`objects_width` задаёт ширину JPEG для детектора (для main-потока 1920 — ставьте 960), `objects_min_h` отсекает
слишком мелкие боксы. Результаты видно прямо на кадре в панели (жёлтые рамки).
Если сервис недоступен, интеграция пишет предупреждение и **автоматически возвращается** к разности кадров.

7.3. Калибровка масштаба (дальние объекты)

В «Детекторе» есть блок **«Калибровка масштаба»**: вводятся эталонный объект (название, реальный размер, размер в
пикселях на кадре) и номерной знак (0,520 м — стандарт РФ). Интеграция считает `px_per_m` как среднее по эталонам,
запоминает ширину/высоту кадра, рост человека (1,70 м по умолчанию) и дату замера.

Кнопка **«Применить по калибровке»** пересчитывает и выставляет:
`objects_tiling` = 1, если человек в модели ≥ 40 px, иначе 2; `objects_min_h` = 0,25 × (рост человека / высота кадра);
`objects_conf` = 0,25.

Пример (пилот «Гостевой 13», 03.11.2026): борт Tundra 92 px при 1,70 м и номер 27,3 px при 0,52 м → **53,3 px/м**;
человек 1,70 м = **91 px (8,4 %)** кадра, 30 px в модели → tiling 2, min_h 0,021. Подробный расчёт — `docs/КАЛИБРОВКА-ДАЛЬНИЕ-ОБЪЕКТЫ.md`.

7.4. Подбор порогов

1. Открыть «Детектор» → «Показать маску»: видно, что именно считается движением.
2. Слишком много ложных: поднять `min_ratio`, `diff_t`, `min_px`, увеличить `confirm_frames`; нарисовать
   зоны-исключения (тени, дорога, качающиеся ветки).
3. Пропускает: снизить `min_ratio`/`diff_t`, повысить чувствительность зоны.
4. Ночь/ИК: отдельный порог `diff_t_ir`, множитель `night_sensitivity`.

---

8. Логика действий и таймеры

```
IDLE ──движение──► ON (таймер hold) ──движение + extend──► ON (таймер сброшен)
  ▲                  │
  │                  ├─ тишина hold ─────► OFF (если включали мы и restore позволяет) ─► IDLE
  │                  ├─ max_on ──────────► OFF (жёсткий лимит) ─► пауза (cooldown)
  └──────────────────┴─ ошибка/выключение ─► OFF
```

Правила:

1. Перед включением проверяются: тумблер интеграции, тумблер зоны, наличие и доступность устройства, дни недели,
   окно времени, «время суток» (закат/до заката), пауза между включениями.
2. Включение — `homeassistant.turn_on` (для `light` с яркостью, если задана); `scene`/`script` — вызов как есть.
3. Запоминается `was_on` — горело ли устройство до нас. По умолчанию **не выключаем** то, что горело раньше.
4. Если человек сам переключил устройство после нашей команды — оно не трогается (карантин 10 минут).
5. Продление: новое движение во время удержания сбрасывает таймер (не чаще раза в 5 с).
   Таймер — **общий на устройство**: если один свет включают несколько зон, удержание

берётся максимальным
из сработавших зон, а лимит `max_on` отсчитывается от первого включения и не отодвигается.

6. Антидребезг: не чаще одной команды в 5 с; жёсткий лимит `max_on`; пауза после выключения.
7. Перезапуск НА: активные таймеры и `was_on` хранятся в `.storage/light_by_video.<entry_id>`; при старте
просроченное гасится (с повторами), действующее — досчитывается. Свет не «зависает» включённым.
7а. **Проверка результата** (как в «Времени света»): после включения/выключения интеграция через 5 с проверяет
фактическое состояние устройства и при расхождении повторяет команду (до 3 раз), затем пишет событие
`skipped` с причиной «устройство не подтвердило переключение».
8. Все решения и пропуски пишутся в журнал (локальное время, зона, доля изменения, причина).

8.1. Расписания (принудительное вкл/выкл)

Вкладка **«Расписания»**: список правил «тип запуска + дни + действие + зона».

Типы запуска (`trigger`):

Тип	Что делает	Поля
`time`	включить/выключить в точное время	`time` (ЧЧ:ММ), `action`, `seconds`
`range`	в начале диапазона **включить** (на `seconds`), в конце — **выключить**	`time_from`, `time_to`, `seconds`; переход через полночь поддерживается
`sunset`	закат ± смещение	`offset_min` (−180…+180), `action`, `seconds`
`sunrise`	рассвет ± смещение	`offset_min` (−180…+180), `action`, `seconds`

Время заката/рассвета берётся у Home Assistant (`homeassistant.helpers.sun.get_astral_event_date`) по координатам конфигурации, поэтому совпадает с `sun.sun`. Смещение: `-30` — за полчаса до заката, `+15` — через 15 минут после.

Тик каждые 20 с; срабатывание идёт через тот же движок (учитываются запреты, «не беспокоить», пауза, «время суток»).
Каждый тик берётся **самый свежий** пропущенный якорь, поэтому после простоя выполняется одно действие, а не очередь.

Догон пропущенного: если НА был выключен в момент срабатывания, при старте событие

выполняется, пока опоздание
меньше `catchup_min` (по умолчанию 360 мин). Допуск всегда не меньше 1 минуты (тик 20 с), поэтому `catchup_min = 0`
означает «только в момент срабатывания». Пропущенные сверх окна помечаются в журнале как пропущенные (`last_run`
обновляется, действие не выполняется).

Ручной запуск — сервис `light_by_video.run_schedule` (`schedule_id`): для диапазона выполняется включение,
для заката/рассвета/времени — действие правила. Ближайшее срабатывание — датчик `sensor.*_next_schedule`
(атрибуты: `schedule`, `action`, `trigger`, `in_sec`, `count`).

Важно: если устройство **уже горело** до срабатывания, интеграция им не управляет (не ставит таймер) — иначе
состояние «залипало». Кнопка/сервис `stop` выключает принудительно, в том числе устройство, включённое вручную
или расписанием.

8.2. Запреты и условия по солнцу

- **Запреты** (вкладка «Запреты»): правило = сущность + атрибут (необязательно) + условие
  (`включён/выключен/больше/меньше/равно/не равно`) + значение + режим. В режиме **«запрещать»** действие не выполняется
  (причина видна в журнале и в ответе сервиса), в режиме **«предупреждать»** — только запись в лог. Недоступная
  сущность запретом не считается.
- **Сенсор «темно»**: по умолчанию `sun.sun`; можно выбрать любой `binary_sensor` (например, датчик освещённости
  с порогом). Поправка к закату в минутах позволяет включать свет **за 15–30 минут до заката** (отрицательное значение)
  или позже.
- Условие зоны «только после заката» использует выбранный сенсор и поправку.

---

9. Сущности, события, API

Сущности: `binary_sensor.<камера>_zona_N` (motion), `binary_sensor.<камера>_source` (connectivity), `sensor.<камера>_last_motion`, `_motions_today`, `_frame_change`; **`sensor.<камера>_status`** (текст: «ожидание
движения» / «включено, осталось N с» / «пауза N с» / «не беспокоить»), **`sensor.<камера>_last_action`** (время

последнего действия/пропуска + атрибуты event/zone/entity/reason), **`sensor.<камера>_next_event`** (когда свет
погаснет автоматически, + `in_sec`); `switch.<камера>_actions`, `switch.<камера>_zona_N_actions`.

Сервисы: `light_by_video.trigger` (`zone_id`, `seconds` — включить вручную), `light_by_video.stop` (`entity` — выключить, без него — все), `light_by_video.reset_cooldown` (`zone_id` — снять паузу), `light_by_video.run_schedule` (`schedule_id` — запустить расписание вручную).

События шины `light_by_video_event`: `detected`, `cleared`, `action_on`, `action_off`, `skipped` (с причиной), `camera_lost`, `camera_back`, `lighting`.

HTTP API (нужен токен HA):

```
GET  /api/light_by_video/state?entry_id=…
GET  /api/light_by_video/frame?entry_id=…[&fresh=1]
GET  /api/light_by_video/mask?entry_id=…            # PNG маски движения
GET  /api/light_by_video/settings
POST /api/light_by_video/settings                    #
zones/detector/enabled/scan_interval/источник/mute_until
POST /api/light_by_video/zone                        # {"action":"add|save|delete", …}
POST /api/light_by_video/test                        # {} | {"action":"on|off","zone_id":…}
GET  /api/light_by_video/events?limit=50
```

---

10. Данные и хранение

Что	Где
Настройки (зоны, детектор, источник)	`config_entry.options` в `/config/.storage/core.config_entries`
Активные таймеры	`/config/.storage/light_by_video.<entry_id>`
Журнал решений	память (200 записей) + события шины
Пароль потока	в настройках записи (как у любой камеры в HA)

---

11. Эксплуатация и диагностика

Симптом	Что смотреть
Не включается	журнал панели: причина (`до заката`, `пауза`, `устройство недоступно`, `вне`

Симптом	Что смотреть
	`окна времени`, `не тот день`)
Ложные срабатывания	«Показать маску», поднять пороги, добавить исключения, включить режим «объекты»
Пропускает	снизить `min_ratio`/`diff_t`, проверить зону, посмотреть `sensor.*_frame_change`
«Камера недоступна»	`binary_sensor.*_source` (атрибуты `source`, `error`), лог HA; поток переоткрывается автоматически при обрыве (`stimeout` 5 с)
⚠ Не закрывать RTSP-контейнер из другого потока	PyAV/ffmpeg не потокобезопасны: `container.close()` из чужого потока роняет Home Assistant (SIGSEGV). Переподключение — только через таймауты чтения; проверено на 0.3.1
Детектор объектов	`GET http://192.168.103.202:8099/health`, `journalctl -u lbv-detector`
Проверка сейчас	кнопка «Проверить сейчас» в панели или `POST /test`

Логи: `docker logs homeassistant | grep light_by_video`; служба детектора — `journalctl -u lbv-detector`.

---

12. Безопасность

- HTTP API интеграции требует авторизации Home Assistant (`requires_auth`).
- Сервис детектора слушает `0.0.0.0:8099` в локальной сети; данные — только кадры, без записи на диск.
  При необходимости ограничить firewall'ом до `192.168.103.200`.
- Пароль RTSP хранится в настройках записи HA (стандартная практика); в логи и на панель он не выводится
  (`rtsp://***@host/...` в состоянии источника).
- Интеграция не требует прав администратора для просмотра панели (`require_admin=False`), но изменение настроек
  доступно только авторизованным пользователям.

---

13. Ограничения

- Разность кадров чувствительна к движущимся теням и листве — зону нужно рисовать там, где ожидается человек.

- YOLOv5n (nano) уверенно видит крупные объекты; мелкие/дальние может пропускать (лечится моделью крупнее).
- Стоящий неподвижно человек не удерживает свет: движение прекращается → таймер истекает (по замыслу).
- CPU-детектор: 175–290 мс на кадр 640×360 (4 потока) — при интервале 1.5 с это ~15–20 % одного ядра.
- Один экземпляр интеграции — одна камера и один набор зон (несколько камер — несколько записей).

---

14. Развитие

1. Несколько камер в одной записи и общие сценарии.
2. Полигональные зоны вместо прямоугольников.
3. Модель объектов крупнее (yolov5s/yolov8s) или класс «животные» отдельно.
4. Уведомления (Telegram/чат центра) о срабатываниях и сбоях.
5. «Умное» выключение: учитывать состояние других датчиков (двери, присутствие).
6. Публикация в HACS-каталог (сейчас — custom repository).

---

15. История версий

Версия	Дата	Главное
0.1.0	03.11.2026	зоны, детектор разности кадров, действия с удержанием, сущности, панель, API; пилот «Гостевой 13»
0.2.0	03.11.2026	4 угла/произвольный размер, вкладка «Источник», «после заката», пауза между включениями, режим «объекты» (YOLOv5n ONNX на ARM), сторож RTSP
0.3.0	03.11.2026	общий таймер на устройство, инференс объектов только при движении, режим «Не беспокоить»
0.3.1	03.11.2026	🔴 убран сторожевой поток RTSP (закрытие ffmpeg-контейнера из другого потока роняло HA с SIGSEGV), вместо него `stimeout`
0.4.0	03.11.2026	запреты по сенсорам, выбор сенсора «темно» + поправка к закату, дробление кадра для дальних объектов, пресеты, боксы на кадре
0.4.1	03.11.2026	кадры «смена света» не рвут серию движения; гашение просроченного включения после рестарта — с повторами
0.5.0	03.11.2026	произвольные (полигональные) зоны с редактированием вершин; маски полигонов

Версия	Дата	Главное
		в детекции и в проверке объектов
0.5.1	03.11.2026	исправлено сохранение вершин (`points` не писались в настройки)
0.6.0	03.11.2026	калибровка масштаба хранится в настройках интеграции (эталон + номер), кнопка «Применить по калибровке» пересчитывает tiling/min_h/conf; блок в панели
0.7.0	03.11.2026	детальная вкладка «Действия» (состояние, таймеры, устройство, расписание, быстрые кнопки) + проверка результата команды с повторами
0.8.0	03.11.2026	датчики «Статус», «Последнее действие», «Ближайшее выключение»; сервисы trigger / stop / reset_cooldown
0.9.0	03.11.2026	расписания вкл/выкл с догоном пропущенного; next_schedule; run_schedule; не управлять уже горящим
0.9.1	03.11.2026	подсказки под каждым полем панели + встроенная вкладка «Справка»
0.10.0	03.11.2026	выбор типа запуска расписания: по времени / диапазон времени / по закату / по рассвету (со смещением)